W4 — React Native ve Expo | Sınav Çalışma Dosyası

Kaynak: zeki-ozen/MobileAppDevelopment / W4
Kapsam: React Native girişi, Expo/Hello Expo, `let` / `const`, props, state, Hook'lar (`useState`, `useEffect`).
Öncelik: Kod parçalarının çoğu **Expo Snack** `App.js` içinde kullanılabilir. Tam uygulama örneklerini `App.js` içine **tek tek** yapıştır.

1. React Native ve Expo nedir?

React Native: JavaScript/React ile Android ve iOS'ta yerel arayüz bileşenleri kullanarak mobil uygulama geliştirme teknolojisi. **Expo:** React Native geliştirmeyi kolaylaştıran araçlar ve servisler; Expo Go, desteklenen projeleri telefonda test etmeye yarar. **Expo Snack:** Tarayıcıda React Native kodu yazıp önizleyebileceğin internet editörü.

Özellik	React (web)	React Native (mobil)
Ekran bileşeni	`<div>`, `<p>`, `<button>`	`<View>`, `<Text>`, `<Button>`
Stil	CSS, CSS Modules	JS stil nesnesi / `StyleSheet`
Tıklama	`onClick`	`onPress`
Metin kutusu	`<input>`	`<TextInput>`
Yerel donanım	Tarayıcı API'leri	Uygun React Native/Expo modülleri

> **W4 sınav ipucu:** `useState` ve `useEffect` mantığı React web ile aynıdır; JSX'te kullanılan **ekran bileşenleri farklıdır**.

2. Expo Snack web editöründe başlama (Windows + iPhone)

1. Bilgisayarda snack.expo.dev aç.
2. `App.js` dosyasını seç; örnek uygulamanın kodunu buraya yapıştır.
3. Gerekli paket varsa Snack `Dependencies` bölümünden ekle.
4. `My Device` görünümündeki QR kodunu iPhone kameranla tara; **Expo Go** ile aç.
5. Kod değiştirip kaydettikçe uygulama önizlemesini güncelle.

Yerel proje (alternatif): Node.js kurulu bilgisayarda `npx create-expo-app@latest` ile proje oluşturup `npx expo start` çalıştırılır. Şablona bağlı olarak ana dosya `App.js` değil `app/index.tsx` veya `src/app/index.tsx` olabilir. **Snack'te ise öğretimde genellikle `App.js` kullanılır.** Windows'ta iOS simulator yerel olarak kullanılamaz, ancak iPhone + Expo Go ile test edebilirsin.

3. En temel `App.js` iskeleti — TAM VE ÇALIŞIR

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.ekran}>
      <Text style={styles.baslik}>Merhaba Expo!</Text>
      <Text>İlk mobil uygulamam.</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  ekran: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#F1F5F9',
  },
  baslik: { fontSize: 26, fontWeight: 'bold', color: '#0F172A' },
});
```

Satırların görevi: `import` gerekli bileşenleri getirir. `App` ana bileşendir. `return (...)` ekranda görünecek JSX'i belirtir. `View` kutudur, `Text` yazıdır. `styles.ekran` stil kurallarını uygular. `export default` uygulamanın ana bileşenini dışa açar.

4. `View`, `Text`, `StyleSheet` ve Flexbox

Stil	Ne yapar?	Sık kullanılan değer
`flex: 1`	Mevcut kullanılabilir alanı doldurur	`1`
`flexDirection`	Çocukların yönü	`'column'` (RN varsayılanı), `'row'`
`justifyContent`	Ana eksende hizalama	`'center'`, `'space-between'`
`alignItems`	Çapraz eksende hizalama	`'center'`, `'stretch'`
`padding` / `margin`	İç/dış boşluk	`12`, `20`
`backgroundColor`	Arka plan	`'#ffffff'`
`borderRadius`	Yuvarlatma	`12`
`fontSize`	Yazı boyutu (`Text`)	`18`

```
<View style={{ flexDirection: 'row', justifyContent: 'space-between' }}>
  <Text>Sol</Text><Text>Sağ</Text>
</View>
```

Tuzak: RN'de `width: '100%'` veya `width: 120` kullan; `width: '120px'` web CSS biçimidir. `Text` dışındaki düz yazılar React Native'de doğrudan çocuk metin olarak gösterilmez; `<Text>` içine yazılmalı.

5. `const` / `let` ve render değişkenleri

```
const baslik = 'Mobil Programlama';  // bağın yeniden atanması yasak
let puan = 5;                        // yeniden atanabilir
puan = puan + 2;
const renkler = ['mavi', 'mor'];
renkler.push('yeşil');               // const dizinin içeriği değişebilir
```

Çok önemli: React bileşeninin içindeki normal `let sayac = 0` değerini artırmak **ekranı güncellemez**. Kalıcı ve görünür şekilde değişmesi için `useState` kullan.

6. Props — Parent'tan çocuğa veri aktarımı

Aşağıdaki tam örnekte `OgrenciKart` bileşenini aynı ekranda farklı verilerle tekrar kullanıyoruz.

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

function OgrenciKart({ ad, not }) {
  return (
    <View style={styles.kart}>
      <Text style={styles.ad}>{ad}</Text>
      <Text>{not >= 50 ? 'Geçti' : 'Kaldı'} ({not})</Text>
    </View>
  );
}

export default function App() {
  return (
    <View style={styles.ekran}>
      <OgrenciKart ad="Ünal" not={85} />
      <OgrenciKart ad="Ayşe" not={45} />
    </View>
  );
}

const styles = StyleSheet.create({
  ekran: { flex: 1, justifyContent: 'center', padding: 24, gap: 14 },
  kart: { padding: 18, backgroundColor: '#DBEAFE', borderRadius: 12 },
  ad: { fontSize: 20, fontWeight: 'bold' },
});
```

`ad="Ünal"` bir string prop'tur; `not={85}` JavaScript sayısıdır. `props` alt bileşende doğrudan değiştirilmez. Değiştirmeyi üst bileşende state ve callback ile yaparsın.

7. State + buton + sayaç — TAM `App.js`

```
import React, { useState } from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';

export default function App() {
  const [sayi, setSayi] = useState(0);
  return (
    <View style={styles.ekran}>
      <Text style={styles.baslik}>Sayaç: {sayi}</Text>
      <Button title="+1 artır" onPress={() => setSayi(x => x + 1)} />
      <Button title="-1 azalt" onPress={() => setSayi(x => x - 1)} />
      <Button title="Sıfırla" onPress={() => setSayi(0)} />
    </View>
  );
}

const styles = StyleSheet.create({
  ekran: { flex: 1, justifyContent: 'center', padding: 30, gap: 16 },
  baslik: { fontSize: 28, fontWeight: 'bold', textAlign: 'center' },
});
```

Sınavda açıklaması: `useState(0)` ilk değeri sıfır yapar. `sayi` okunacak değerdir. `setSayi` günceller. `onPress` butona dokunulunca fonksiyon çağırır. `setSayi(x => x + 1)` önceki state'e göre güvenli artırma sağlar.

8. TextInput ile kontrollü form — TAM `App.js`

```
import React, { useState } from 'react';
import { View, Text, TextInput, Button, Alert, StyleSheet } from 'react-native';

export default function App() {
  const [ad, setAd] = useState('');
  const [yas, setYas] = useState('');

  function gonder() {
    if (!ad.trim() || !yas.trim()) {
      Alert.alert('Uyarı', 'İsim ve yaş zorunludur.');
      return;
    }
    Alert.alert('Kayıt', `${ad}, ${yas} yaşındasın.`);
  }

  return (
    <View style={styles.ekran}>
      <Text style={styles.baslik}>Kayıt Formu</Text>
      <TextInput
        style={styles.girdi}
        placeholder="Adını yaz"
        value={ad}
        onChangeText={setAd}
      />
      <TextInput
        style={styles.girdi}
        placeholder="Yaşını yaz"
        keyboardType="number-pad"
        value={yas}
        onChangeText={setYas}
      />
      <Button title="Kaydet" onPress={gonder} />
    </View>
  );
}

const styles = StyleSheet.create({
  ekran: { flex: 1, justifyContent: 'center', padding: 24, gap: 15 },
  baslik: { fontSize: 24, fontWeight: 'bold' },
  girdi: { borderWidth: 1, borderColor: '#94A3B8', borderRadius: 10, padding: 12 },
});
```

`TextInput` her zaman metin (`string`) döndürür; sayısal klavye `keyboardType="number-pad"` olsa da sayıya dönüştürme gerekebilir: `Number(yas)`. `trim()` boşluk girilmesini boş kabul eder. `Alert.alert()` iPhone'da da uyarı penceresi gösterir.

9. `useEffect` ve zamanlayıcı — TAM `App.js`

```
import React, { useEffect, useState } from 'react';
import { View, Text, Button } from 'react-native';

export default function App() {
  const [saniye, setSaniye] = useState(0);
  const [calisiyor, setCalisiyor] = useState(false);

  useEffect(() => {
    if (!calisiyor) return;
    const id = setInterval(() => setSaniye(x => x + 1), 1000);
    return () => clearInterval(id);
  }, [calisiyor]);

  return (
    <View style={{ flex: 1, justifyContent: 'center', padding: 30, gap: 12 }}>
      <Text style={{ fontSize: 26 }}>Geçen süre: {saniye} sn</Text>
      <Button title={calisiyor ? 'Durdur' : 'Başlat'}
              onPress={() => setCalisiyor(x => !x)} />
      <Button title="Sıfırla" onPress={() => setSaniye(0)} />
    </View>
  );
}
```

Satır mantığı: `useEffect` `calisiyor` değişince çalışır. Açıkken 1 saniyelik timer başlar. Return içindeki `clearInterval` eski timer'ı temizler; timer birikmesini önler. Hook'u koşul içinde değil, effect'in içindeki koşulla kullanıyoruz.

10. Sık karşılaşacağın kod kalıpları

İstediğin şey	Kopyalanacak kalıp
Ekrana yazı koy	`<Text>Merhaba</Text>`
Bir kutu oluştur	`<View style={{padding: 20}}>` … `</View>`
Buton ekle	`<Button title="Bas" onPress={() => Alert.alert('Basıldı')} />`
State tanımla	`const [ad, setAd] = useState('');`
Metin al	`<TextInput value={ad} onChangeText={setAd} />`
Şarta göre göster	`{gorunur && <Text>Görünüyorum</Text>}`
İki yazıyı yan yana koy	`flexDirection: 'row'`
Ekranı ortala	`flex:1, justifyContent:'center', alignItems:'center'`
Listeye yeni kayıt ekle	`setListe(eski => [...eski, yeni]);`
Koşula göre renk	`backgroundColor: aktif ? 'green' : 'gray'`

11. Yaygın hatalar ve çözümleri

Hata	Neden / çözüm
`Text strings must be rendered within a <Text>`	Düz yazıyı `<Text>` içine al
`Can't find variable: useState`	`import { useState } from 'react'` unutulmuş
Buton çalışmıyor	`onPress={() => islem()}` veya `onPress={islem}` kullan
Yazı görünmüyor	`Text` yerine `View` içine düz metin yazılmış olabilir
State değişiyor ama ekran değişmiyor	Doğrudan `state = ...` yerine setter kullan
Foto görünmüyor	`Image` için genişlik/yükseklik veya doğru `source` gerekli
`className` beklediğin gibi çalışmıyor	Saf React Native'de `style` / `StyleSheet` kullan
`.map` liste elemanları hata veriyor	Döndürülen öğelere kararlı `key` ekle

12. W4 öz değerlendirme soruları

S1. Expo Snack ile Expo Go farkı? **Cevap:** Snack tarayıcıda kod editörüdür; Expo Go desteklenen projeyi telefonda açıp test etmeye yarar.

S2. React web'deki `<div>` mobilde ne olur? **Cevap:** `<View>`.

S3. `const [sayi, setSayi] = useState(0)` içinde `sayi` ve `setSayi` nedir? **Cevap:** State değeri ve setter.

S4. `onPress={fonksiyon()}` neden istenmeyebilir? **Cevap:** Fonksiyonu render anında çalıştırabilir; olay sırasında çalışması için `onPress={fonksiyon}` ver.

S5. `useEffect` cleanup niçin önemli? **Cevap:** Timer, listener ve diğer kaynakları temizlemek, tekrarlandıklarında çoğalmalarını önlemek için.

S6. `props` alt bileşen tarafından doğrudan değiştirilmeli mi? **Cevap:** Hayır.

S7. Flexbox'ta React Native varsayılan yön? **Cevap:** `column`.

S8. Sayısal klavyeli TextInput'un değeri number mıdır? **Cevap:** Hayır, state'te string tutulur; gerekiyorsa dönüşüm yapılır.

13. Depoyla eşleştirme

Depo dosyası	Bu rehberde
`1-react-native-intro.md`	1–4, 11
`2-example-hello-expo.md`	2–3
`3-react-native-variable-const.md`	5
`4-react-native-props.md`	6
`5-react-native-state.md`	7–8

Depo dosyası	Bu rehberde
`6-react-native-hooks.md`	9

Kaynak: W4 klasörü.
Bu çalışma dosyası eğitim amaçlı özgün Türkçe özet ve örnek kodlardan oluşur; Expo SDK sürümü ve şablon değişebileceği için mevcut proje dosya yapını kontrol et.